VLB entry changes: go-live plan, revision 2

Decisions of 2 November 2026, the added release 4, the meter cross-check report, and live Equipment master synchronization between Pulse and Pulse-II

Caliber Pulse MIS | Supersedes docs/vlb-entry-changes-go-live-plan-2026-11-02.pdf sections 1 and 10; sections 5 to 9 of that document still apply | Repository main at commit d7486f8 | Prepared 2 November 2026

1. Decisions taken

Six gaps were discussed after the review. The decisions below are final for this plan. Releases 1 to 3 from the first plan are unchanged; this revision adds release 4, a reporting change, and a separate integration project for Pulse.

Gap	Decision	Where it lands
1 Tipper coverage check before Lock VLB	Deferred. Not built now; revisit after the team has run release 4 for a month.	Backlog
2 Duplicate protection for manual cards	Build.	Release 4, change 10
3 Meter continuity across shifts	Not at entry or in scanning. A silent cross-check in reporting shows where opening does not match the previous closing, so the team can see which drivers report wrong readings.	Release 4, change 12 (report)
4 Pulse writes recorded as local-only	Replace with real writes. Anything created or transferred in Pulse-II is done in Pulse at once, and the Equipment master is kept in live synchronization both ways.	Pulse sync project, section 5
5 Late batch alerts	Build, with Telegram to the site Project Manager and the production head.	Release 4, change 11
6 Staging slot	No.	Dropped

> One open point from the discussion: the message said "point 6 no" and then "photo 6, please add and make it live". Only five screenshots were attached and none shows a sixth item. This plan treats the staging slot as dropped. If the sixth item meant the two smaller items at the end of the discussion, the per-batch count of automatic versus manual postings and the keyboard path to the next tipper, say so and they go into release 4 as changes 13 and 14; both are small.

2. Release 4: duplicate rule, late batch alerts, meter report

Change 10. One manual VLB card per tipper per shift, with an audited override

Files: log-book.mjs (insertLogBook, update path), vlb-batch-policy.mjs (duplicate query under the existing batch lock), src/log-book-form.jsx (409 handling, override dialog), vlb-batch-format.mjs

Behaviour after the change

- Before a manual vehicle card is inserted or edited, the server looks for another active vehicle card with the same site, book date, shift and tipper. Scanned cards keep their own unique index on card number; this rule covers every card, manual or scanned, by tipper.
- When one exists the save is refused with 409: "Tipper <door> already has card <VLB number> for <site>, <date>, shift <n>. Open that card to correct it, or save a second card with a reason." The message names the existing card so the user can open it from the batch register.
- The form offers "Save as additional card" which asks for a reason of at least ten characters. The second save carries the reason; the server stores it on the card as duplicateReason and writes card_added_duplicate to the batch audit with both card numbers. No second card is ever saved silently.
- The batch report marks additional cards with a flag column so the reviewer sees them, and the machine-wise totals still include their trips.

• The check runs inside the same advisory lock that batch membership uses, so two users saving the same tipper at the same instant get one card and one 409.

Tests

• test/log-book.test.mjs: second manual card for the same tipper and shift returns 409 naming the first; with a reason it saves and the audit row exists; a different shift or site is unaffected; editing the first card does not trip the rule against itself. test/vlb-batch-report.test.mjs: additional-card flag rendered.

Schema: None. duplicateReason lives in the card JSON; the audit table already exists.

Risk if wrong: Sites where a tipper legitimately fills two cards per shift must give a reason each time. If that becomes frequent the reason list can be made a drop-down later.

Change 11. Late batch alerts to the site Project Manager and the production head by Telegram

Files: vlb-batch-alerts.mjs (new worker and rules), vlb-batches.mjs (schema addition), shift-master.mjs (shift end times already present), otp-delivery.mjs (reuse the Telegram outbox sender), mis-telegram.mjs (outbox and links already present), server.mjs (register the worker), src/mis-admin.jsx (recipient settings)

Behaviour after the change

• A minute worker, modelled on the retention cleanup worker, computes for every operational site and every Shift Master row in force today the shift end time in India time. Two deadlines follow: created-by, shift end plus two hours; closed-by, shift end plus six hours. Both offsets are settings, editable in MIS administration, with these defaults.

• If no batch exists for the site, date and shift at created-by, the alert "No VLB batch created" goes out. If the batch exists but is not closed at closed-by, the alert "VLB batch still open" goes out with the stage (VLB open or ELB open), card count, and the owner login.

• Recipients: every linked Telegram chat of a user who is a Project Manager assigned to that site (the same pmLogins lookup the OTP flows use), plus the production head. The system has no production head role today; see the open question in section 7. Until it is answered the alert uses a settings list of production head logins, maintained in MIS administration, and falls back to the site MIS users when the list is empty.

• Messages go through the existing Telegram outbox with an event key per site, date, shift and alert type, so each alert is sent once even if the worker runs on two instances or restarts. A reminder repeats every two hours until the condition clears, each with its own event key. WhatsApp is not used for these alerts.

• Message text, in English, for example: "Pulse-II: Sasti OC, 2 Nov 2026, shift II. VLB batch still open (ELB open, 94 cards, owner mis.sasti) 6 h after shift end. Open Site batches to close it." Hindi follows the chatbot language preference of the recipient when one is stored.

• The dashboard VLB card gains a "batches due today" strip per site: created, VLB locked, closed, with the same colours used by the batch list. This is read-only and uses the batch list API already in place.

Tests

• test/vlb-batch-alerts.test.mjs (new): deadline computation across midnight shifts and the dated Shift Master revision; no alert before the deadline; one outbox row per event key; reminder cadence; recipient resolution for PM at site, production head list, fallback; no alert for dates before the operational start or for a deleted batch.

Schema: One additive table mis_vlb_batch_alert_state(site, book_date, shift, alert_type, first_sent_at, last_sent_at, cleared_at) so reminders and clearing are tracked; settings stored in app_settings under vlb_batch_alerts.

Risk if wrong: A wrong Shift Master end time produces alerts at the wrong hour. The alert text includes the shift end it used, so a wrong master row is visible and correctable.

Change 12. Meter continuity report: silent cross-check of opening against previous closing

Files: meter-continuity-report.mjs (new query and rules), hierarchy-report-catalogue.mjs (new entry), src/card-reports.jsx or the saved VLB report page (new report tab), report-xlsx.mjs and table-export-pdf.mjs (export reuse)

Behaviour after the change

- Nothing changes at entry or in scanning. Cards save exactly as today.
- A report, filtered by site, date range and optional tipper or driver, lists every vehicle card alongside the previous active card of the same tipper (by date and shift order, across days). Columns: date, shift, tipper, driver on the card, previous card number and date, previous closing KMR, this opening KMR, KMR gap, previous closing HMR, this opening HMR, HMR gap, card source (manual or scan), card number.
- Rows are classed: match (gap zero), tolerance (gap within a site setting, default 2 km and 0.5 h), reading below previous closing (negative gap), jump above the tolerance, and no previous card. Colours match the existing report conventions.
- A second tab summarises by driver: cards, matches, negative readings, jumps, average absolute gap, so the team can see which drivers repeatedly report readings that do not follow from the previous shift. A third tab summarises by tipper for meter faults.
- Export to Excel and PDF through the shared report controls. The report is available to MIS users and Project Managers for their sites and to administrators for all sites.
- When the team later wants a hint at entry time, the same rule module can be called from the form; that is deliberately not part of this release.

Tests

- test/meter-continuity-report.test.mjs (new): previous-card selection across shift order and days, deleted cards skipped, class boundaries at the tolerance, driver and tipper summaries, site scope enforced.

Schema: None. Tolerances live in app_settings.

Risk if wrong: A tipper whose card was missed in one shift shows a jump in the next; the report says "no previous card" only when none exists at all, so the missed shift shows as a jump. That is useful information, not a false alarm, and the deferred coverage check (gap 1) is the eventual answer.

Discussion of the reporting approach for meters

Moving the check out of entry and into reporting is the right call for how the cards are produced. The MIS user is copying what the driver wrote; a warning at entry would be answered by typing what the card says anyway, and in scanning it would add one more brown card to clear. A report over a week or a month shows something no single card can: whether a driver, or a tipper, keeps producing readings that do not follow from the shift before. That is a behaviour and maintenance finding, and it belongs with the people who act on it, the site PM and production head, not with the person at the keyboard.

Two things make the report trustworthy. First, it must pick the previous card by shift order, not by save time, because scans arrive late and corrections are made days after. The handover module already orders a tipper's cards by date and shift for trip links, and the report reuses that ordering. Second, a tolerance is needed because a tipper may travel between the parking area and the first loading point after the previous card closed; the default of two kilometres and half an hour is a starting point for the team to adjust per site.

What the report cannot do is tell the difference between a driver misreading a meter and a meter that was replaced or reset. The tipper summary tab is there for that: a tipper with negative gaps across several drivers is a meter problem, a driver with negative gaps across several tippers is a reading problem.

3. Release 4 acceptance script

Script E: duplicate rule, alerts, meter report

1. Save a manual card for tipper T1 in shift II. Save a second manual card for T1 in shift II. Expect: 409 naming the first card number; the form shows "Save as additional card".

2. Choose it without a reason. Expect: refused. Enter a reason. Expect: saved, batch audit shows card_added_duplicate, batch report flags the second card.

3. Edit the first card. Expect: saves without a duplicate complaint.

4. Set the created-by offset to one minute in MIS administration, pick a site with no batch for the current shift, wait for the worker. Expect: one Telegram message to each linked site PM and to each production head login; no second message within the reminder interval; the dashboard strip shows the shift as not created.

5. Create and close the batch. Expect: no further alert; the alert state row shows cleared_at.

6. Open the Meter continuity report for the site and the last seven days. Expect: a row per card with previous closing, opening and gap; a negative reading appears in the negative class; the driver tab counts it under that driver.

7. Restore the alert offsets to two and six hours.

4. Release schedule with release 4

Release	Changes	Depends on	Build and test	Live verification
R1	1, 2, 3, 4 plus the CRLF test fix	Nothing	2 to 3 working days	Scripts A and B
R2	5A or 5B	Team decision on auto-save	Half a day or 1 day	Script C
R3	6, 7, 8, 9	Decision on blocking negative meters	3 to 4 working days	Script D
R4	10, 11, 12	Production head recipients (section 7); Shift Master end times verified per site	3 to 4 working days	Script E
Pulse sync	Section 5	Pulse repository access and a Pulse service credential	See section 5.6	Section 5.7

Release 4 can be built in parallel with release 3 because they touch different files. The Pulse sync project is independent of all four releases and should start now, because of the finding in section 5.1.

5. Live Equipment master synchronization between Pulse and Pulse-II

5.1 Where things stand, and why this is urgent

• Pulse at pulse.cmll.in is the only Equipment master. Pulse-II reads it with an authorised Pulse administrator session through GET /api/masters every sixty seconds, matches records through pulse_master_links, and replaces each linked record's data with Pulse's copy. No call writes to Pulse. This is documented in ERP-LOCAL.md and implemented in pulse-master-sync.mjs.

• The express transfer and the express equipment creation added with the two-stage batch write only to Pulse-II because no Pulse write endpoint exists. They record pulseWrite.mode = local-only.

• Consequence confirmed in code: an express transfer updates currentLocation on the local Equipment master record, and the next sync, within sixty seconds, overwrites that record with Pulse's copy, where the asset is still at the old site. The site hold on the card comes back. Only the Vehicle transfers record keeps the completed status. Express-created equipment is not linked to any Pulse id, so it survives the sync but exists nowhere in Pulse.

• So today the express OTP flow gives the operator a transfer that disappears a minute later. Until the write path exists, the honest interim is section 5.5.

5.2 Target design

Pulse stays the master. Every change that Pulse-II makes to an asset is made in Pulse first, inside the same user action, and Pulse-II applies it locally only after Pulse has confirmed. Changes made in Pulse reach Pulse-II within seconds through a push from Pulse, with the existing sixty-second poll kept as the safety net. The result is one Equipment master with two front ends, not two masters.

Direction	Mechanism	Latency	Failure behaviour
Pulse-II to Pulse (writes)	Two authenticated endpoints in Pulse: complete vehicle transfer, create equipment. Called synchronously by Pulse-II inside the OTP completion.	Immediate	Pulse unreachable or refuses: the OTP completion fails with a clear message and nothing changes in either system. The operator retries later; the OTP stays valid for its remaining minutes.
Pulse to Pulse-II (reads)	Pulse posts a signed change notice to Pulse-II whenever an Equipment master record changes, including changes made through the two new endpoints. Pulse-II fetches the changed record at once. The sixty-second full poll remains.	Seconds; at most sixty seconds if the notice is lost	Notice lost: the poll catches up. Pulse-II down: notices are retried by Pulse for an hour, then the poll catches up.

5.3 Work in the Pulse repository

Pulse is a separate application and repository; this repository holds only Pulse-II. The following is built there and deployed to pulse.cmll.in before Pulse-II can switch on writes.

1. A service account of a new type, "Integration", that cannot sign in through the browser and holds exactly two permissions: complete vehicle transfer, create equipment. Its token is long-lived, stored only in the Pulse-II App Service settings, and revocable from Pulse administration.

2. POST /api/integrations/vehicle-transfers/:id/complete. Body: idempotency key (the Pulse-II OTP request id), approving PM login, requesting MIS login, approval channel, approved-at time. Pulse runs its own three-step transfer in one transaction with those identities recorded, updates currentLocation and lastTransferDate on the asset, writes its audit trail, and returns the asset. A repeated call with the same idempotency key returns the same result without acting twice.

3. POST /api/integrations/equipment. Body: idempotency key, the normalized Equipment master fields Pulse-II collected, site, approving PM, requester. Pulse validates against its own duplicate rules (door, registration, chassis), creates the record with createdVia express-otp, and returns the new Pulse id. Same idempotency rule.

4. An outbound change notice: on any Equipment master insert or update, Pulse posts {masterName, sourceId, changedAt} to the Pulse-II URL configured in Pulse administration, signed with a shared secret (HMAC over the body and a timestamp). Pulse retries with backoff for one hour and shows the last delivery status in administration.

5. Pulse administration pages: integration accounts, change-notice target and secret, and a log of integration calls with the idempotency key, caller, result and time.

6. Tests in the Pulse repository for both endpoints, idempotency, permission scope, duplicate rules and the notice retry.

5.4 Work in Pulse-II (this repository)

1. pulse-writer.mjs (new): implements completeTransfer and createEquipment against the two endpoints, with the service token from PULSE_INTEGRATION_TOKEN, a ten-second timeout, and the idempotency key taken from the OTP request id. It returns {mode: pulse, pulseId, at} or throws with a user-facing message. Both express modules already accept a pulseWriter; wiring it in server.mjs is a one-line change each.

2. Order of operations inside the OTP completion transaction: verify OTP, call Pulse, then apply locally with the Pulse id linked in pulse_master_links, then commit. If the local apply fails after Pulse succeeded, the idempotent retry on the next attempt re-reads Pulse's result and completes locally without a second Pulse change.

3. POST /api/pulse/change-notices (new): verifies the HMAC and timestamp window, then triggers a targeted fetch of that record from Pulse and applies it through the existing applyPulseSnapshot path for a single record. Unknown or stale notices are ignored and logged.

4. A write mode setting PULSE_WRITE_MODE with values off, dry-run, live. off is today's behaviour (local-

only, refused by the UI with the message in 5.5). dry-run calls a validate variant of each endpoint, records what Pulse would have done, and still refuses the local change. live performs the write.

5. Equipment created or transferred through the express flow gets a "Pulse" badge with the Pulse id in the batch holds panel and in the Equipment master page, and a "pending in Pulse" badge while in off or dry-run mode.

6. Replay of the backlog: a one-time administration action lists every express transfer and creation with pulseWrite.mode = local-only, in date order, and replays each through the writer with its original idempotency key. Each replay result is recorded on the record. Transfers already completed in Pulse by hand are detected by Pulse's own state and returned as already-complete.

7. Tests: writer success, timeout, 409 from Pulse, idempotent retry, change-notice signature and replay; express modules refuse completion when the writer throws; the sync no longer reverts a transferred asset because Pulse now agrees.

5.5 Interim behaviour until Pulse ships the endpoints

- Switch express completion to refuse in off mode with the message "Pulse cannot be updated from here yet. Ask the Pulse administrator to complete transfer <id> in Pulse; the card stays on hold and clears automatically after the next sync." This is honest about what happens today and stops the one-minute reversal from confusing operators.
- Add the "Pending in Pulse" list to MIS administration now, from the local-only records, with a daily Telegram reminder to the Pulse administrator while the list is non-empty. This list becomes the replay queue in 5.4 step 6.
- This interim is a half-day change in Pulse-II and should ship with release 4.

5.6 Sequence and effort

Step	Where	Effort	Gate to the next step
Interim refuse and Pending in Pulse list	Pulse-II	Half a day	Ships with release 4
Integration account and the two endpoints	Pulse	3 to 4 working days	Pulse tests green; endpoints reachable from Pulse-II with the service token
Writer, write mode setting, change-notice receiver	Pulse-II	2 to 3 working days	Dry-run against live Pulse returns validate results for five real pending transfers
Change notices from Pulse	Pulse	1 to 2 working days	Pulse-II shows a Pulse edit within ten seconds
Replay of the backlog	Pulse-II, run by the Pulse administrator	One hour	Pending in Pulse list empty
Switch PULSE_WRITE_MODE to live	Pulse-II App Service setting, no deploy	Minutes	One express transfer at one site completed end to end and visible in both systems

5.7 Verification and rollback

- Verification: from a batch hold, raise an express transfer; the source-site PM enters the OTP; within ten seconds the asset shows at the new site in Pulse administration and in the Pulse-II Equipment master; the hold clears without any further action; the audit in both systems names the PM and the MIS user. Repeat for an express creation. Then edit an asset in Pulse and watch it change in Pulse-II within ten seconds.
- Rollback: set PULSE_WRITE_MODE to off. Express completion returns to the interim refusal. Assets already written to Pulse are correct in both systems and need no action. Change notices can be disabled in Pulse administration; the poll continues.
- Security: the integration token never reaches the browser; it is read from the App Service setting at request time. The change-notice secret is separate from the token. Both are rotated from Pulse administration and the Pulse-II settings together.

6. Deployment, note to users, rollback

Sections 5 to 9 of the first plan apply unchanged: branch per release, pull request, green CI, local walk-through, deploy between shifts through the main workflow, health check at pulse-ll.cmll.in, revert on main as rollback. Release 4 adds two lines to the note to users: a second card for the same tipper in a shift now needs a reason, and site PMs will receive a Telegram message when a shift's batch is late.

7. Open questions to settle before building release 4

Question	Why it matters	Proposed answer
Who is the production head for alerts?	No such role exists. Manager roles are Project Manager, Production Manager, Maintenance Manager, MIS Manager, each assigned per site.	A settings list of logins in MIS administration, one or more, applying to all sites; or, if one person per region, a Production Manager with region scope. Decide which; the code supports a list from day one.
Alert offsets	Two and six hours after shift end are guesses.	Keep as defaults, editable in settings; ask each site MIS user what their normal closing time is after the first week of data.
Is a second card per tipper per shift ever legitimate?	Decides whether the override in change 10 is a reason box or a hard block.	Reason box, audited, flagged in the report. Review after one month; if reasons are always the same two or three, make them a drop-down.
Meter tolerance per site	Two kilometres and half an hour default.	Settings per site; start with the default everywhere.
Pulse repository access	Section 5.3 is built there.	Confirm who owns the Pulse repository and deployment, and that the integration account type is acceptable to the Pulse administrator.
The sixth item	See the callout in section 1.	Confirm whether the two small items join release 4.